CHAPTER 2

BASICS OF C#

Learning Objectives

2.1 INTRODUCTION TO C#

Microsoft .net was formerly (in earlier times) known as Next Generation Windows Services (NGWS). This is a platform for developing the next generation of Windows/Web applications. These applications would transcend device boundaries and fully harness the power of the internet. However, this platform required a language, which could take its full advantage. This is one of the facts that led to the development of C#. C# has evolved from C/C++. Hence, it retains its family name. The # (hash symbol) in musical notations is used to refer to a sharp note and is called "Sharp", hence the name is pronounced as "C sharp".

C# is an elegant and type-safe object-oriented language that enables developers to build a variety of secure and robust applications that run on the .NET Framework. You can use C# to create traditional Windows client applications, XML Web services, distributed components, client-server applications, database applications, and much more. Visual C# provides an advanced code editor, convenient user interface designers for building Windows and Web applications, a compiler, integrated debugger, and many other tools to make it easier to develop applications.

The C# compiler is considered to be the most efficient compiler in the .net family and a major part of the .net base classes libraries (which constitutes a major part of .net) itself are written in C#. Moreover, C# could be considered as a modern replacement for C/C++ languages. It gives access to many of the facilities previously available only in C++.

It is important to look C# not just as a programming language but as an integral part of the .net platform. The .net platform revolutionizes facilities available for Windows programming. It provides benefits like the automatic garbage collector for automatically cleaning up resources occupied by dead objects, and enhanced libraries that cover areas ranging from Windows GUI support to data access to generating ASP.net pages.

C# syntax is highly expressive, yet it is also simple and easy to learn. The curly-brace syntax of C# will be instantly recognizable to anyone familiar with C, C++ or Java. Developers who know any of these languages are typically able to begin to work productively in C# within a very short time. C# syntax simplifies many of the complexities of C++ and provides powerful features such as nullable value types, enumerations, delegates, lambda expressions and direct memory access, which are not found in Java. C# supports generic methods and types, which provide increased type safety and performance, and iterators, which enable implementers of collection classes to define custom iteration behaviors that are simple to use by client code. Language-Integrated Query (LINQ) expressions make the strongly-typed query a first-class language construct.

As an object-oriented language, C# supports the concepts of encapsulation, inheritance, and polymorphism. All variables and methods, including the Main method (the application's entry point) are encapsulated within class definitions. A class may inherit directly from one parent class, but it may implement any number of interfaces. Methods that override virtual methods in a parent class require the override keyword as a way to avoid accidental redefinition. In C#, a struct is like a lightweight class, it is a stack-allocated type that can implement interfaces but does not support inheritance.

In addition to these basic object-oriented principles, C# makes it easy to develop software components through several innovative language constructs, including the following:

- Encapsulated method signatures called *delegates*, which enable type-safe event notifications.
- *Properties*, which serve as accessors for private member variables.
- *Attributes*, which provide declarative metadata about types at run time.
- Inline *XML* documentation comments.
- *Language-Integrated Query (LINQ)* which provides built-in query capabilities across a variety of data sources.

The key point of C# however is that it enhances developer productivity and increase safety, by enforcing strict type checking. It features a garbage collector, which relieves the programmer of the burden of manual memory management. C# offers extensive interoperability. The managed environment is appropriate for most enterprise applications.

Some applications do require "Native code", either for performance reasons or to interoperate with existing Application Programming Interfaces (APIs). In such situations developers use C++ even when they would prefer to use a more productive and easier language. For example, the

Scanned with OKEN Scanner

Windows API is written in C/C++. This forces the developers to use Visual C++ for system programming, even though they would prefer to use Visual Basic. C# addresses these problems by providing the simplicity and the productivity of Visual Basic while providing native support for the Component Object Model (COM) and Windows-based APIs. If you have to interact with other Windows software such as COM objects or native Win32 DLLs, you can do this in C# through a process called "Interop." Interop enables C# programs to do almost anything that a native C++ application can do. C# also allows restricted use of native pointers and the concept of "unsafe" code for those cases in which direct memory access is absolutely critical.

The C# build process is simple compared to C and C++ and more flexible than in Java. There are no separate header files, and no requirement that methods and types be declared in a particular order. A C# source file may define any number of classes, structs, interfaces, and events.

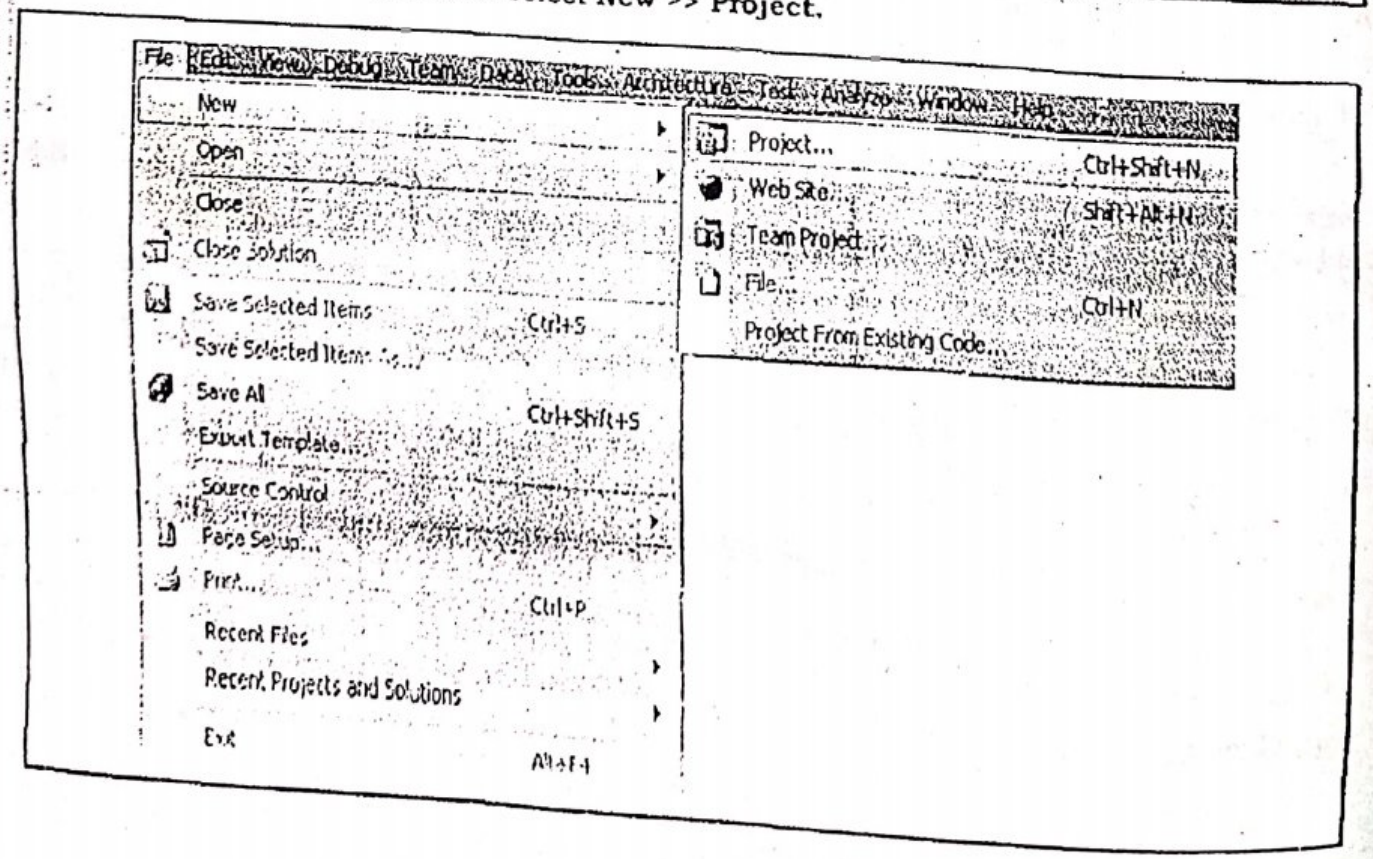
2.1 CREATING A CONSOLE APPLICATION IN C#

The .NET Framework supports console applications, graphical user interface (GUI) applications (Windows Forms), browser-based applications (Web Forms and ASP.NET), and Web Services. This chapter will focus on command line applications, which are known as console applications. Console applications have a text-only user interface. To understand the flow of a C# program. let us begin by analyzing a very simple program in **C# Console Application.**

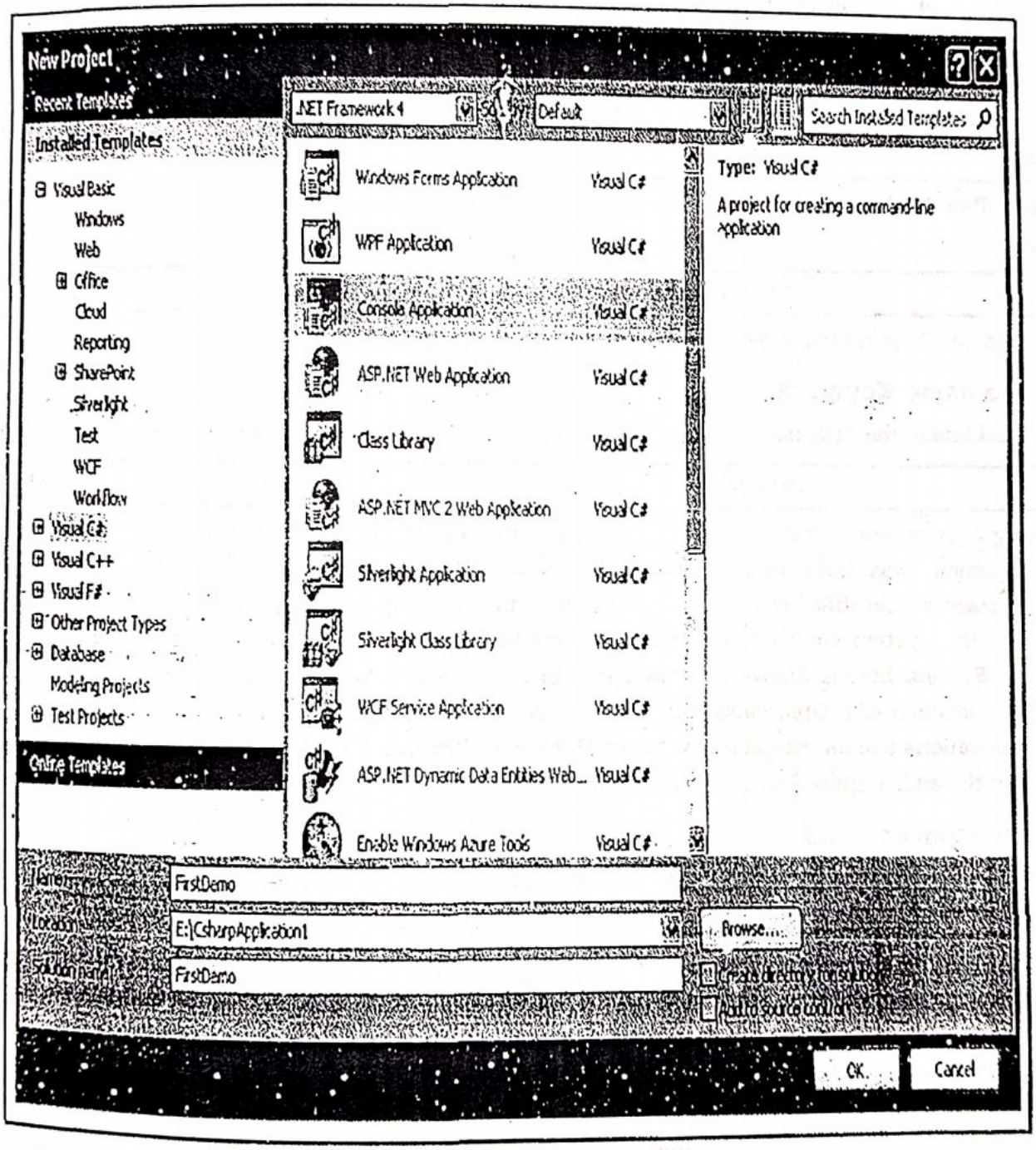
Program 2-1

Description of the Program 2-1: This program will simply display a message on the user's screen.

Step-1: Click on **File** menu and select **New >> Project.**



Step-2: In **New Project Dialog**, select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application.**



Step-3: Name of the Application will be given in **Name** textbox. Here it is given as **"FirstDemo"**. This is actually the name of the **Namespace.**

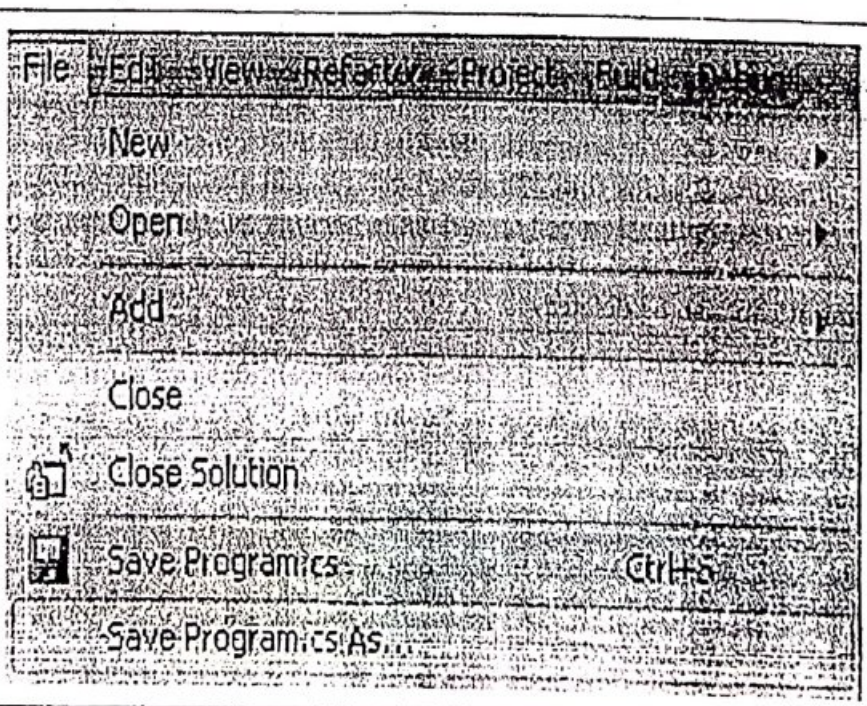
Step-4: Set the location of the application in **Location** textbox and click on **Ok** button. Here it is "**E:\CsharpApplication1**".

Note: After this, You can check a directory will be created into the given location. Here, in **E:\CsharpApplication1** a directory named **"FirstDemo"** will be created which contains other required files for the application.

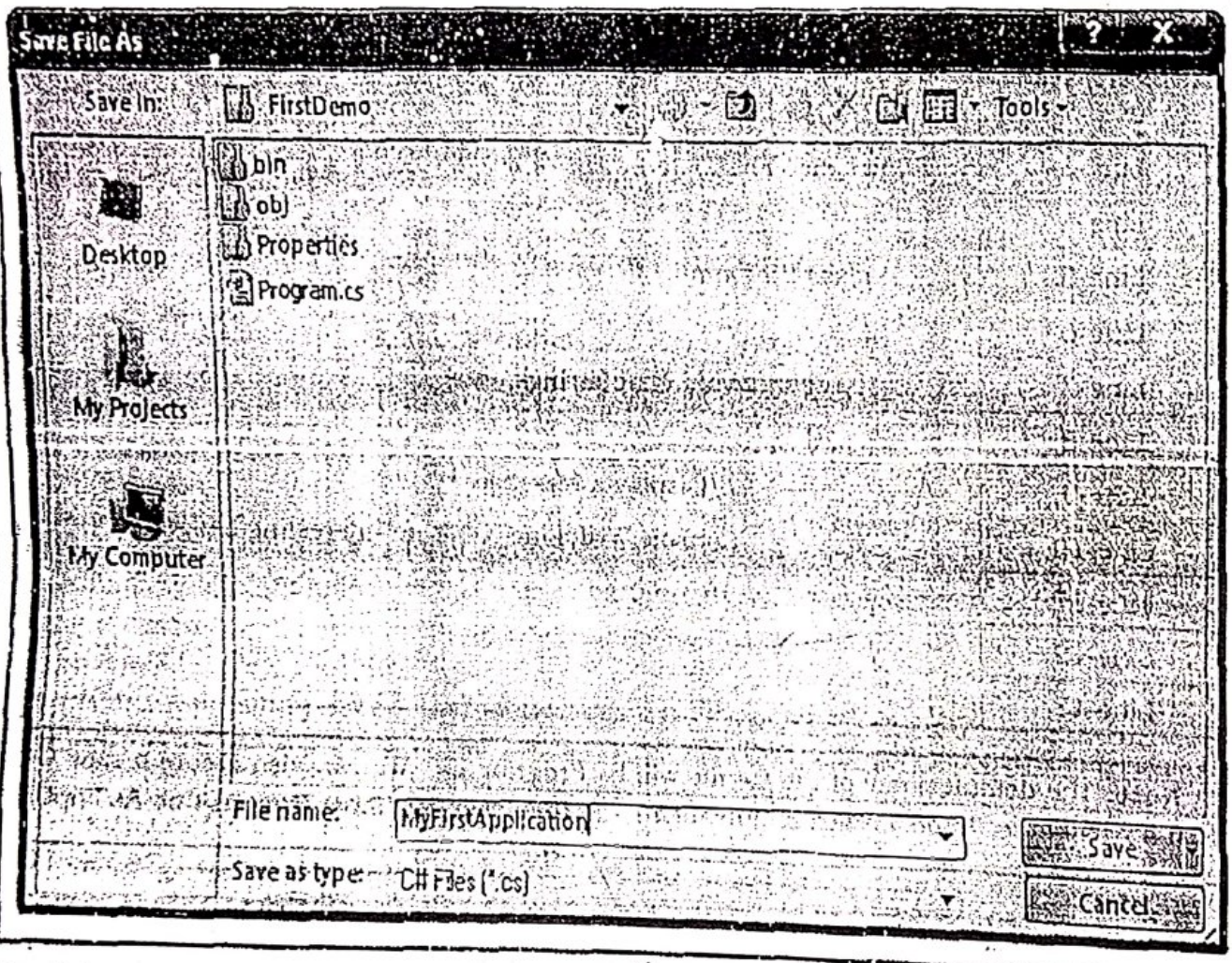
Step-5: Add the following code to the C# Console Application.

Line Number	Code
Line-1	/* This is a simple program in C# */
Line-2	using System;
Line-3	namespace FirstDemo
Line-4	{
Line-5	class Program
Line-6	{
Line-7	static void Main(string[] args)
Line-8	{
Line-9	//Display a Message
Line-10	Console.WriteLine("C# is an Interesting Language!");
Line-11	}
Line-12	}
Line-13	}

Note: The default name of the C# file will be **Program.cs**. We can also create a new C# file with a new name. To do so click on File menu and then click on **"Save Program.cs As"** option.



The **"Save File As"** dialog will appear and now, we can save our C# file with the new name given by the programmer. Here it is named as **"MyFirstApplication"**.



By doing so, a new C# file, named **"MyFirstApplication.cs"** will be created into the **"FirstDemo"** directory. Besides this, **"Program.cs"** file will also be there, but now the solution will work for **"MyFirstApplication.cs"** only. This is because all the references of the application will be set to the **"MyFirstApplication.cs"**. It means that **"Program.cs"** will have no effect on the solution and it can also be deleted. .

Step-6: To see the output of the above code **Press Ctrl + F5.** The following window will appear

This will be the final output of the application. Here's the end of **Program 2-1.**

2.2.1 Understanding a Console Application in C#

Here, we are going to describe the various code statements used in **Program 2-1** (written in **Step-5**) to understand the structure of a C# console application.

2.2.1.1 Comments

Comments are the programmer's text to explain the code. All the comments are ignored by the language compiler and are not included in the final executable code. C# uses syntax for comments that is similar to C/C++ and Java. The text following double slash marks (//any comment) are **Line Comments**. These types of comments end with the end of the line. For example:

Line-9	//Display a Message

C# also supports **Comment Block**. These comments are begins with "/*" and ends with "*/". For example:

Line-1	/* This is a simple program in C# */

Comments can be included in any part of a C# program.

2.2.1.2 The using Keyword

The second line of the code is:

Line-2	using System;

The **using System** statement is quite similar to the **#include** statement used in C/C++. The #include statement was used to include another header (source) file, so as to make the functions, present within that header file, a part of the current program. Similarly the keyword **using** imports the **System** class file and makes the methods present within it as a part of the program. But **System** here is known as **namespace** and not as a **header file**. A namespace is just a logical collection of related classes in C#. The **System** namespace contains the classes that most applications use for interacting with the **Operating System**. The classes that are used more often are the ones required for basic Input/Output.

2.2.1.3 Namespaces in C#

When working on a huge project it is quite obvious that you will be creating a lot of classes. It becomes equally obvious that sometimes the names of these classes might clash. There are two ways to solve this problem. The first alternative is that we can rename the classes so that they no longer clash. But it is undesirable and hard to remember names. The second alternative is to use **namespaces.**

A namespace is just a logical collection of related classes in C#. The sole purpose of using namespaces is to avoid conflicts. Making use of namespaces in your code will reduce the complexities when you want to reuse your code in some other application. You just cannot do without namespaces in C#. In the example discussed so far we have use the System namespace. As mentioned earlier, the **System** namespace contains all classes that are required to interact with the Operating System, including the console (your screen) and keyboard.

So, in the third line of our program, we are declaring that the following classes (within { } block) are part of **FirstDemo** namespace.

```
Line-3    namespace FirstDemo
Line-4    {
          ..... //Add classes to the namespace FirstDemo
Line-13   }
```

The namespaces may contain classes, events, exceptions, delegates and other namespaces called "Internal namespaces". These internal namespaces can be defined as follows:

```
namespace External
{
    namespace Internal
    {
        ..... //Add classes to the namespace Internal
    }
}
```

2.2.1.4 Classes in C#

All of our C# programs contain at least one class. The **Main()** method resides in one of the classes. Classes consist of data and functions which are also known as fields and methods respectively. For example:

```
Line-5    class Program
Line-6    {
          //Declaration of Fields
          // Methods
Line-12   }
```

2.2.1.5 The Main() Method

The Main() method of the application is defined inside **Program** class.

```
Line-5    class Program
Line-6    {
Line-7        static void Main(string[] args)
Line-8        {
                  ..... //Body of the Main() method
Line-11       }
Line-12   }
```

The Main() method is the entry point of a C# program. This means that the execution of our C# program begins from beginning of the Main() method and terminates with the termination of the Main() method. The Main() method is designated as **static** as it will be called by the Common Language Runtime (CLR) without making any object of our **Program** class. The Main() method is also declared **void** as it does not return anything. **Main** is the standard name of this method, while the **string[] args** is the list of parameters that can be passed to main while executing the program from command line.

Note: Ma
wa
•
•
•
•
•
}
Similarly,

2.2.1.6 Basi

Basic Input
System names
- Console.
- Console.

We have alre

```
Line-10
```

Console.Writ
parameter to dis

Syntax-1

Example:

Syntax-2:

Example:

Syntax-3:

Or

Example:

Here, {0} is

Scanned with OKEN Scanner

Note: Main() method starts with capital 'M'. A Main() method can be defined in number of ways, such are as follows:

- static void Main(String[] args)
- public static void Main(String[] args)
- static void Main()
- public static void Main()
- static int Main(String[] args)

```
{
        .....
        return 0;
}
```

Similarly, you can use many variations of Main() method.

2.2.1.6 Basic Input Output in C#

Basic Input and Output operations are performed in C# using the **Console** class in the **System** namespace. The two widely used methods are:

- Console.WriteLine()
- Console.ReadLine()

We have already used Console.WriteLine() in our program as follows:

```
Line-10        Console.WriteLine("C# is an Interesting Language!");
```

Console.WriteLine() method is used to display data on the screen. It takes a string as its parameter to display on the console window. It can be used in various ways, such are as follows:

Syntax-1: To display only a Message.

```
Console.WriteLine("Message to be displayed");
```

Example:

```
Console.WriteLine("Hello World!");
```

Syntax-2: To display only value of a Variable.

```
Console.WriteLine(variable);
```

Example:

```
int x=20;
Console.WriteLine(x);
```

Syntax-3: To display a message with value of a Variable.

```
Console.WriteLine("Message {variable position}",variable);
```

Or

```
Console.WriteLine("Message "+variable);
```

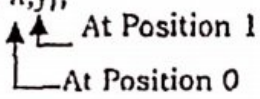
Example:

```
int x=20;
Console.WriteLine("Value of X is "+x);
Console.WriteLine("Value of X is {0}", x);
```

Here, {0} is the position of variable x. To understand it, see following example:

Example: int x=20,y=30;

Console.WriteLine("Value of Y is {1} and X is {0}", x,y);



The output of the statement will be **"Value of Y is 30 and X is 20"**

As we use Console.WriteLine() method to display data on the screen, we can use Console.ReadLine() mehod to get input from the user. The ReadLine() method reads all characters upto a carriage return. The input is returned as a string.

Syntax: Variable=Console.ReadLine();

Example:
```
string name;
Console.Write("Enter your Name : ");
name = Console.ReadLine();
Console.WriteLine("Your Name is {0}", name);
```

2.3 LANGUAGE FUNDAMENTALS OF C#

In this section, we'll introduce statements and expressions, the building blocks of any program. You'll learn about variables and constants, operators, naming convention, Arrays. This is all very basic stuff, but it's the foundation you need to start getting fancy. Without variables, your applications can't actually process any data. All variables need types, and variables are used in expressions. You'll see how neatly this all fits together. It includes:

- Basic Data Types and their mapping to CTS
- Variables
- Constant Variables or Symbols
- Naming Conventions for Variables and Methods or Identifiers
- Operators
- Operators Precedence
- Flow Control Statements
- Arrays

2.3.1 Basic Data Types and their mapping to CTS

Like any programming language, C# defines an intrinsic set of fundamental data types, which are used to represent local variables, member variables, return values, and parameters. Unlike other programming languages, however, these keywords are much more than simple compiler-recognized tokens. Rather, the C# data type keywords are actually shorthand notations for full-blown types in the System namespace. **Table 2-A** lists each system data type, its range, the corresponding C# keyword, and the type's compliance with the common language specification (CLS).

Scanned with OKEN Scanner

Note: CLS-compliant .NET code can be used by any managed programming language. If you expose non-CLS-compliant data from your programs, other languages may not be able to make use of it.

Table 2-A

C# Type (Keyword)	CLS Compliant ?	System Type (.Net Type)	Size in Bytes	Description
Integral Types				
byte	Yes	System.Byte	1	Unsigned 8-bit number, ranges from 0 to 255.
sbyte	No	System.SByte	1	Signed 8-bit number, ranges from–127 to + 128
short	Yes	System.Int16	2	Signed 16-bit number, ranges from –32768 to +32767
ushort	No	System.UInt16	2	Unsigned 16-bit number, ranges from 0 to 65535
int	Yes	System.Int32	4	Signed 32-bit number, ranges from –2147483648 to +2147483647
unit	No	System.UInt32	4	Unsigned 32-bit number, ranges from 0 to 4294967295
long	Yes	System.Int64	8	Signed 64-bit number, ranges from –9223372036854775808 to +9223372036854775807
ulong	No	System.UInt64	8	Unsigned 32-bit number, ranges from 0 to 18446744073709551615
Floating Point Types				
float	Yes	System.Single		32-bit floating-point number, ranges from $\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$ with 7 digit precision. Requires the suffix 'f' or 'F'.
double	Yes	System.Double		64-bit floating-point number, ranges from $\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$ with 15-16 digit precision.
Other Types				
bool	Yes	System.Boolean	1	Represents truth or falsity.
char	Yes	System.Char	2	Contains any single Unicode character enclosed in single quotation mark such as 'a'.

	Yes	System.Decimal	12	96-bit signed number, ranges from ± 1.0 × 10^{28} to ± 7.9 × 10^{28} with 28-29 digits precision. Requires the suffix 'm' or 'M'.
string	Yes	System.String	–	Represents a set of Unicode characters, range is limited by system memory.
object	Yes	System.Object	–	This is the base class of all types in the .NET universe. We can store any data type in an object variable.

C# provides us with an **Unified Type System**. This means that all data types in C#, whether t will be a reference type or a value type, are derived from just one class, the **Object** class. It simply means that the Object class is the ultimate base class for all data types. It means that literally everything in C# is an object since all types are derived from the Object class. This can be demonstrated by the **Program 2-2.**

Program 2-2

Description of the Program 2-2: This program will demonstrate that everything is an object in C#. In this program we are going to convert a number into a string using ToString() method.

Step-1: Click on **File** menu and select **New >> Project.**

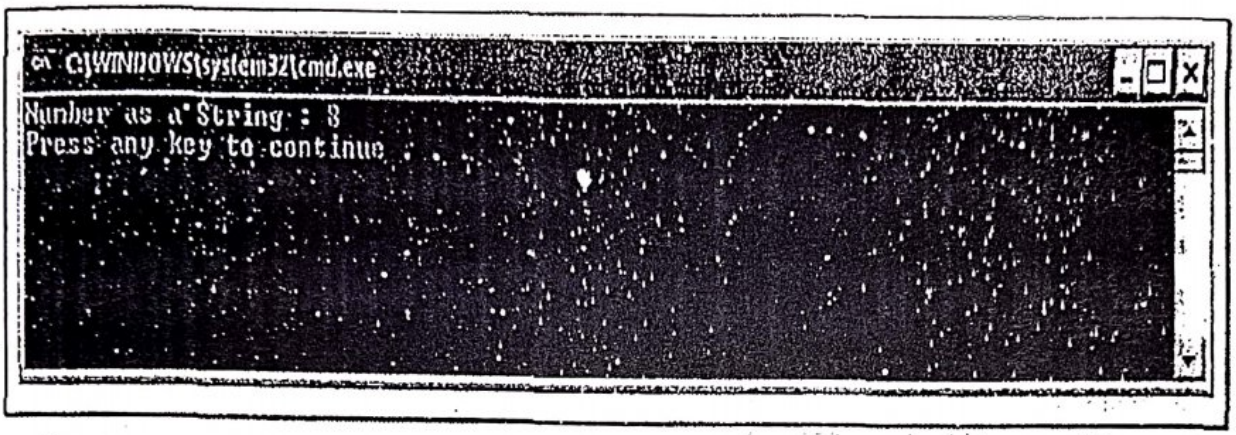
Step-2: In **New Project Dialog,** select **Project types** as **Visual C#** and from **Visual Studio installed Templates** Select **Console Application.**

Step-3: Name the Application **"Everything_Is_Object"** and set location.

Step-4: Add the following code to the C# Console Application.

```
using System;
namespace Everything_Is_Object
{
    class Program
    {
      static void Main(string[] args)
      {
            //Declare a string variable
            string CheckThis;
            //Using number(8) as object and convert it into string
            CheckThis = 8.ToString();
            Console.WriteLine("Number as a String : {0}", CheckThis);
      }
    }
}
```

Step-5: Press Ctrl + F5. The output window will appear as follows:



Here's, the end of the **Program 2-2.**

Observe the line of code in bold **(CheckThis=8.ToString();)**. It is something we have not come across in any programming language. After all 8 is a number how can it have a method? But no! This is the C# realm, where **everything is an object.** Even the number 8. It has methods of which we have shown one, the **ToString()** method. This method converts the number to a string. The above code will be executed without any errors and will store the number 8 as string in the variable **CheckThis** as expected. One more thing about C# is that since all types are derived from a common class (namely Object) they all have similar characteristics.

2.3.2 Variables

During the execution of the program, data is temporarily stored in memory. A variable is the name given to the memory location holding a particular type of data. So, each variable has associated with it a **data type** and a **value.** Declaration of a variable does three things:

- It tells the compiler what the variable name is.
- It specifies what type of data the variable will hold.
- The place of declaration decides the scope of the variables.

In C#, Variables are declared as follows.

Syntax: **DataType** VariableName;
Example: **int** x;

The above statement will reserve the area of 4 bytes in memory to store an integer type value, which will be referred to **in** the rest of the program by identifier '**x**'.

You can also initialize the variable as you declare it and can also declare/initialize multiple variables of the same type in a single statement.

Examples: **int** x=10;
double pi=3.14, r=2.5;

In C#, you must declare variable before using them. Also, there is the concept of **"Definite Assignment"** in C#. Definite Assignment says, "Local variables (variables defined in methods) must be initialized before being used". This can be demonstrated by the **Program 2-3.**

Program 2-3

Description of the Program 2-3: *This program will demonstrate the concept of "Definite Assignment". In this program, first, we will try to display the value of a local variable 'x' without assigning any value to it. See what will happen.*

Step-1: Click on **File** menu and select **New >> Project.**

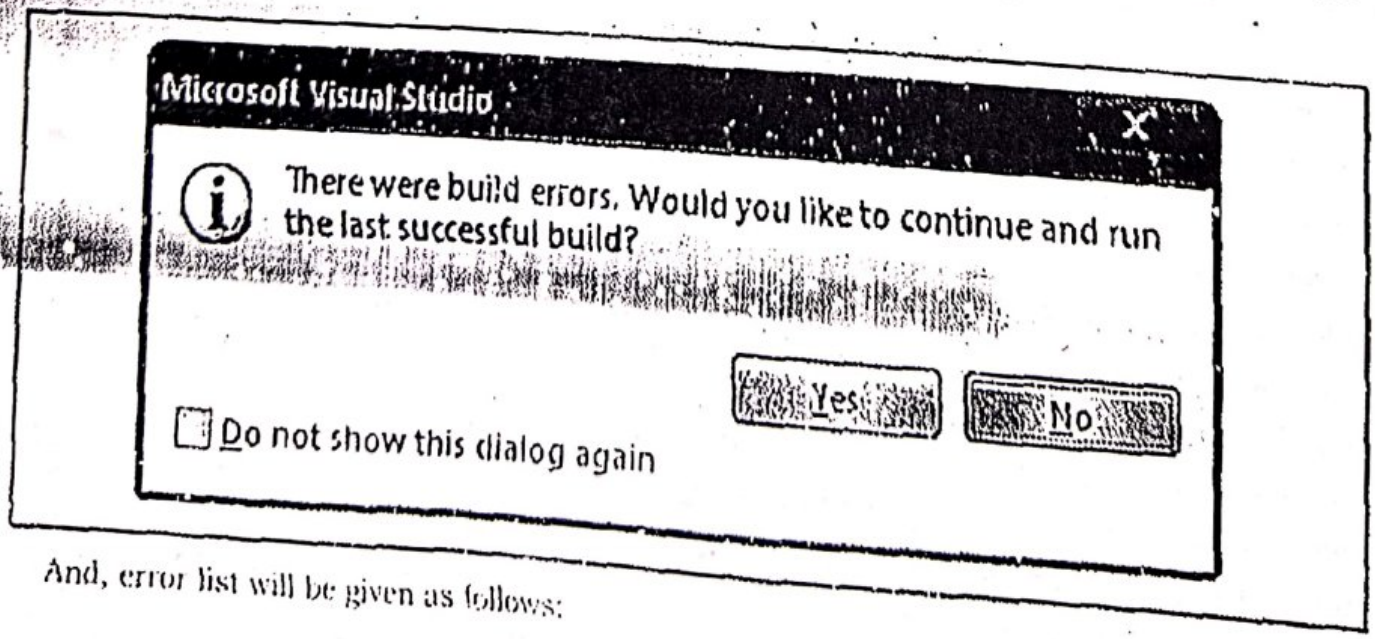
Step-2: In **New Project Dialog**, select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application.**

Step-3: Name the Application **"DefiniteAssignment"** and set location.

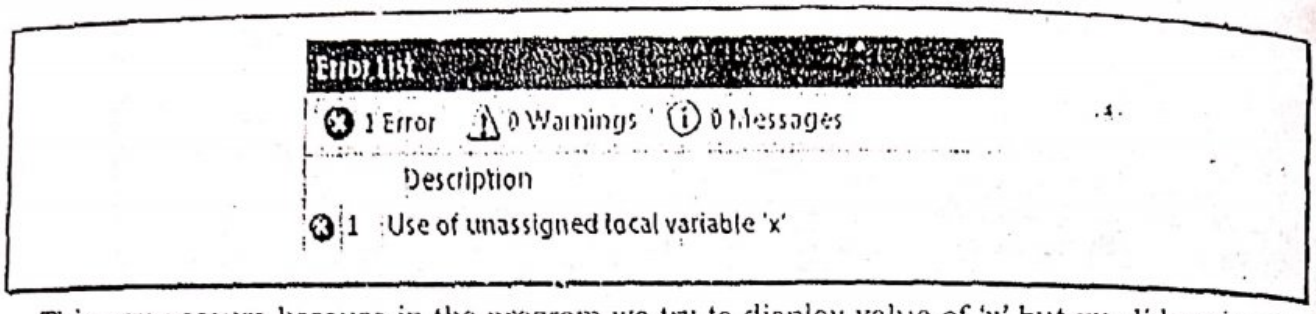
Step-4: Add the following code to the C# Console Application.

```
using System;
namespace DefiniteAssignment
{
        class Program
        {
            static void Main(string[] args)
            {
                //Declaring a local Variable.
                int x;
                //Trying to display value of the local variable 'x'
                System.Console.WriteLine(" X = {0}".x);    //error
            }
        }
}
```

Step-5: Press Ctrl + F5. The following window will appear due to error. Click on **"No"** button



And, error list will be given as follows:



This error occurs because in the program we try to display value of 'x' but we did not assign any value to the variable 'x'.

Step-6: To make it correct, add the following line of code after the declaration statement and before the display statement of the program.

```
x=10;     //Assignment Statement
```

Step-7: Now, again **Press Ctrl + F5.** The output window will appear as follows:

This is the final output of the program. Here's, the end of the **Program 2-3.**

This shows that C# does not assign default values to the local variables. C# is also type safe language which means that value of a particular data type can be stored only in their respective data type. We cannot store a double type value in an integer type of variable.

For example: The following statement is invalid in C#.

```
int x=2.5;
```

Here, variable 'x' is an integer type variable and its value 2.5 is of double type so, there will be an error in this statement.

2.3.3 Constant Variables or Symbols

Constants are the variables, whose values, once defined at compile time, can never be changed. Constants are declared using the **const** keyword.

Syntax: const DataType ConstantName = Value;

Example: const double PI=3.14;

Constants must be initialized as they declared. It is a syntax error to write as follows:

```
const double PI;
```

or

```
const double PI;
PI=3.14;
```

It is conventional to use capital letters while naming constants.

2.3.4 Naming Conventions for Variables and Methods or Identifiers

An identifier can be the name of any variable, method, enumeration etc. Microsoft suggests to use **Camel Notation** (first letter in lowercase) for variables and Pascal Notation (first letter in uppercase) for methods. Each word after the first word in the name of the both variables and methods should start with a capital letter. For example, Variable names following Camel Notation could be: area, netProfit, cashRecieved, studentName, etc.

Simillarly, some of the names of methods following Pascal notation are: GetData(), Display(), Calculate(), WriteLine(), Start(), etc.

Some of the **rules** for naming variables and methods in C# are listed below:

- Name of the identifier is case-sensitive.

  Example: int a, A; //Statement is valid in a C# program.

- Length of the name of the identifier must not be longer than 512 characters.
- Name of the identifier must start with an alphabet, underscore (_), or @.

  Examples:
  ```
  int age;          //valid variable name
  int _age;         //valid variable name
  int @age;         //valid variable name
  int #age;         //invalid variable name
  void _Show()      //valid method name
  void @Show()      //valid method name
  void 1Show()      //invalid method name
  ```

- White spaces are not permitted.

  Examples:
  ```
  double simple interest;   //invalid variable name
  void Display Data()       //invalid method  name
  ```

- Subsequent characters may be letters, digits or underscore characters.

  Examples:
  ```
  double simple_interest;   //valid variable name
  double simple@interest;   //invalid variable name
  void Display_Data()       //valid method name
  void Display@Data()       //invalid method name
  ```

- When choosing a name for your identifier, use full words instead of cryptic abbreviations.

  Examples: double p,r,t,si; //valid but not recommended
  double principal, rate, time , simpleInterest; //valid and recommended

- Name you choose must not be a Keyword or reserved word.

  Examples:
  ```
  int if;          //invalid variable name. 'if' is a keyword.
  void while()     //invalid method name. 'while' is a keyword.
  ```

2.3.5 Operators

C# supports a rich set of operators. An operator is a symbol that tells the computer to

perform certain mathematical or logical manipulations. Operators are used to perform manipulations on operands. C# operators can be categorized as follows:

- Arithmetic operators
- Assignment operators
- Relational operators
- Logical operators
- Unary operators
- Conditional operator
- Bitwise operators

2.3.5.1 Arithmetic Operators: These operators are used to perform general mathematical operations on operands such as addition, subtraction, multiplication and division. These can be explained as follows:

Operator	Description	Example
+	Used to add two numbers as well as to concatenate two strings.	int result = 5 + 10 ; Value of **result** will be : **15** string str = "Good" + "Day" Value of **str** will be : **Good Day**
–	Subtract two numbers.	int result = 55 + 10; Value of **result** will be : **45**
*	Multiply two numbers.	int result = 3*7 ; Value of **result** will be : **21**
/	Divide two numbers. Rule : Second operand must be non zero.	int result = 23/4 ; Value of **result** will be : **5**
%	Used to find remainder Rule : Second operand must be non zero.	int result = 45%7 ; Value of **result** will be : **3** double result = 12%2.5 ; Value of **result** will be : **2**

The use of all above mentioned arithmetic operators can be demonstrated by a **Program 2-4.**

Program 2-4

Description of the Program 2-4: *In this program we will take two numbers from the user as input and will perform all arithmetic operations on these two numbers.*

Step-1: Click on **File** menu and select **New >> Project.**

Step-2: In **New Project Dialog**, select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application.**

Step-3: Name the Application **"ArithmeticOperators"** and set location.

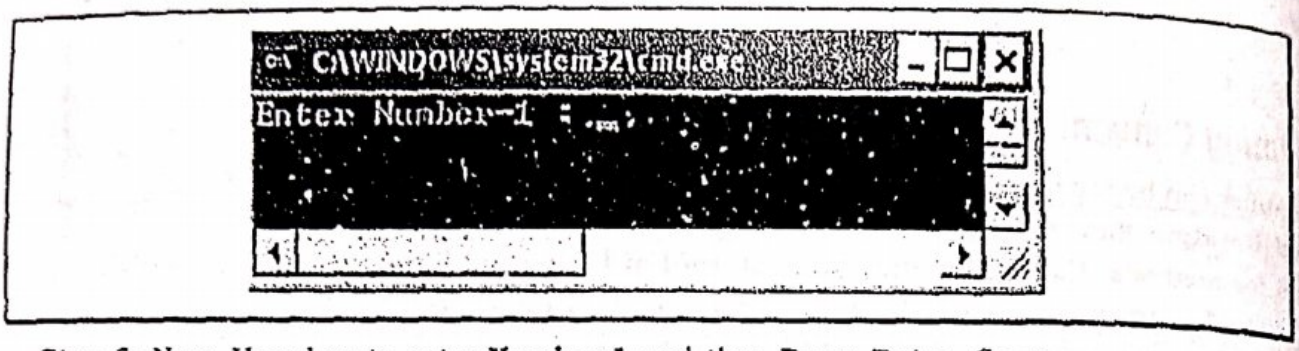
Step-4: Add the following code to the C# Console Application.

```
using System;
```

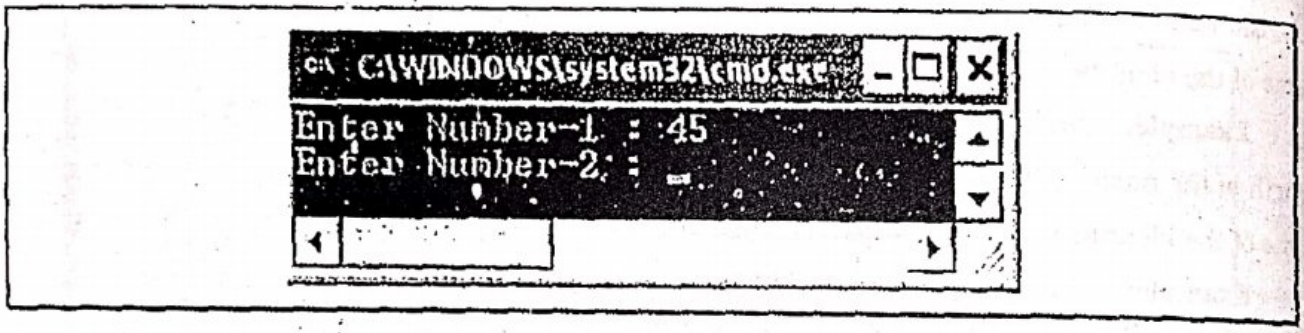
```
namespace ArithmeticOperators
{
    class Program
    {
        static void Main(string[] args)
        {
            //Declaration of required variables
            int num1, num2;
            int add,subtract,multiply,divide,remainder;
            string str1, str2, concatenation;

            //Taking User Input as String
            Console.Write("Enter Number-1 : ");
            str1 = Console.ReadLine();
            Console.Write("Enter Number-2 : ");
            str2 = Console.ReadLine();

            //Converting Strings into integers
            num1 = Convert.ToInt32(str1);
            num2 = Convert.ToInt32(str2).

            //Use of arithmetic operators
            add = num1 + num2;
            subtract = num1 - num2;
            multiply = num1 * num2;
            divide = num1 / num2;
            remainder = num1 % num2;
            concatenation = str1 + str2;

            //Displaying results of all aritmetic operations
            Console.WriteLine("Sum is : {0}", add);
            Console.WriteLine("Subtraction is : {0}", subtract);
            Console.WriteLine("Multiplication is : {0}", multiply);
            Console.WriteLine("Division is : {0}", divide);
            Console.WriteLine("Remainder is : {0}", remainder);
        Console.WriteLine("Concatenation is : {0}", concatenation);
        }
    }
}
```

Scanned with OKEN Scanner

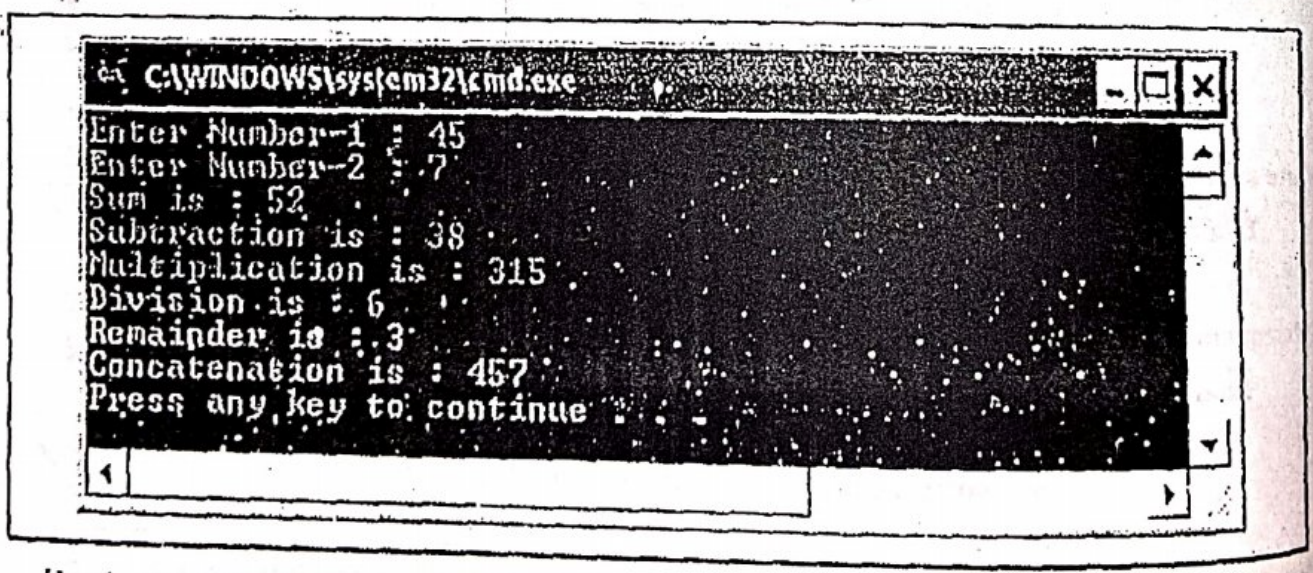
Step-5: Press Ctrl + F5. The output window will appear as follows:



Step-6: Now, User has to enter **Number-1** and then **Press Enter.** Suppose user enters **45** and presses enter then, the output window will demand to enter **Number-2** and will be look like as follows:



Now, User has to enter **Number-2** and then **Press Enter.** Suppose user enters 7 and presses enter then, the result of various arithmetic operations will be displayed on output window and will be look like as follows:



Here's, the end of the **Program 2-4.**

2.3.5.2 Assignment Operators

Assignment operator (=) is used to assign value to a variable or an object. There are some shorthand assignment operators (+=, -=, *=, /=, %=) that we may use in our program. These are as follows:

Operator	Description	Example
=	Simple assignment.	int number 10 ; Value of **number** will be : **10**

+=	Additive assignment.	int number = 10;
		number += 5;
		Value of **number** will be : **15**

-=	Substractive assignment.	int number = 10;
		number -= 5;
		Value of **number** will be : **5**

*=	Multiplicative assig nent.	int number = 10;
		number *= 5;
		Value of **number** will be : **50**

/=	Division assignment.	int number = 10;
		number /= 5;
		Value of **number** will be : **2**

%=	Modulo assignment.	int number = 19;
		number %= 5;
		Value of **number** will be : **4**

The use of all these operators can be demonstrated by a **Program 2-5**.

Program 2-5

Description of the Program 2-5: *In this program we will perform all kinds of assignments on a number and display their results.*

Step-1: Click on **File** menu and select **New >> Project.**

Step-2: In **New Project Dialog,** select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application.**

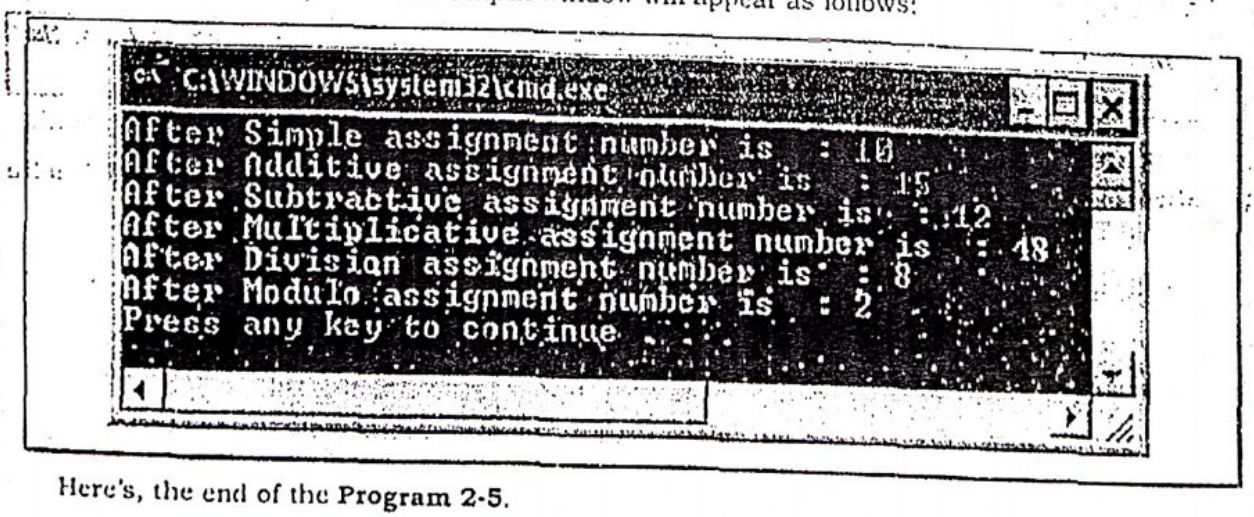
Step-3: Name the Application **"AssignmentOperators"** and set location.

Step-4: Add the following code to the C# Console Application.

```
using System;
namespace AssignmentOperators
{
  class Program
  {
    static void Main(string[] args)
    {
        //Declaration of required variables
        int number;
```

```
        /* Use of assignment and shorthand assignment operators and display
           their result.
         */

         //Use of Simple Assignment Operator (=)
         number = 10;
        Console.WriteLine("After Simple assignment number is : {0}", number);

         //Use of Various Shorthand Assignment Operators
         number += 5;    //equivalent to: number=number+5;
        Console.WriteLine("After Additive assignment number is : {0}", number);

     number -= 3;    //equivalent to: number=number-3;
    Console.WriteLine("After Subtractive assignment number is : {0}", number);

    number *= 4;    //equivalent to: number=number*4;
    Console.WriteLine("After Multiplicative assignment number is : {0}", number);

    number /= 6;    //equivalent to: number=number/6;
    Console.WriteLine("After Division assignment number is : {0}", number);

    number %= 3;    //equivalent to: number=number%3;
    Console.WriteLine("After Modulo assignment number is : {0}", number);
     }
  }
}
```

Step-5: Press Ctrl + F5. The output window will appear as follows:



Here's, the end of the **Program 2-5.**

2.3.5.3 Relational Operators

Relational operators are also called as **Comparison Operators**. These operators are used to perform comparisons in conditional statements. Relational operators always result in a Boolean value i.e. either **True** or **False**.

The most commonly used relational operators are:

Operator	Description	Example

<	Less than	int a = 10, b = 20;
		bool res = a < b;
		Value of **res** will be : **True**

>	Greater than	int a = 10, b = 20;
		bool res = a > b;
		Value of **res** will be : **False**

<=	Less than or equal to	int a = 10, b = 10;
		bool res = (a <= b);
		Value of **res** will be : **True**

>=	Greater than or equal to	int a = 10, b = 20;
		bool res = (a >= b);
		Value of **res** will be : **False**

==	Equality operator, used to check equality.	int a = 10, b = 20;
		bool res = (a==b);
		Value of **res** will be : **False**

!=	Equality operator, used to check un-equality.	int a= 10, b = 20;
		bool res = (a != b);
		Value of **res** will be : **True**

The use of all these operators can be demonstrated by a **Program 2-6.**

Program 2-6

Description of the Program 2-6: ***In this program we will take two numbers from the user as input and will perform various comparisons on these two numbers.***

Step-1: Click on **File** menu and select **New >> Project.**

Step-2: In **New Project Dialog**, select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application**.

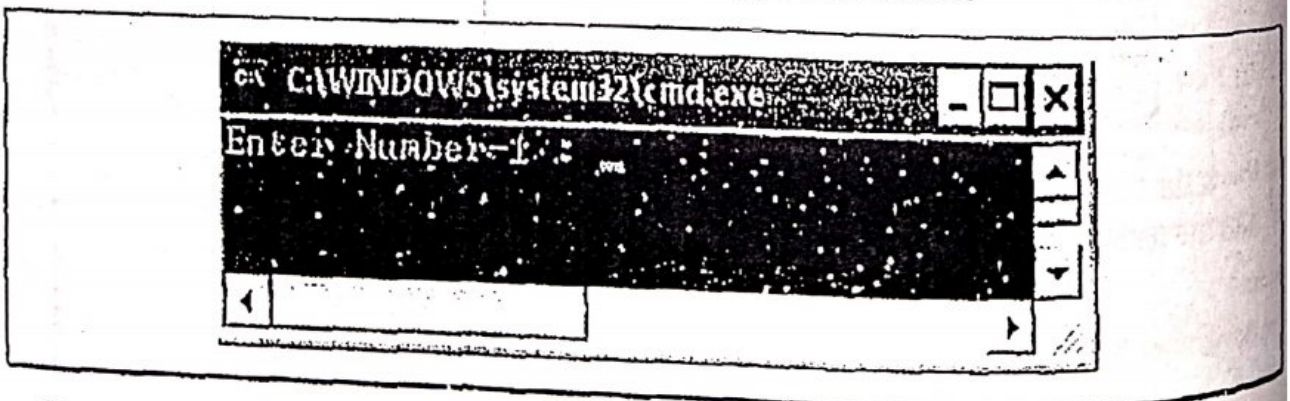
Step-3: Name the Application "**RelationalOperators**" and set location.

Step-4: Add the following code to the C# Console Application.

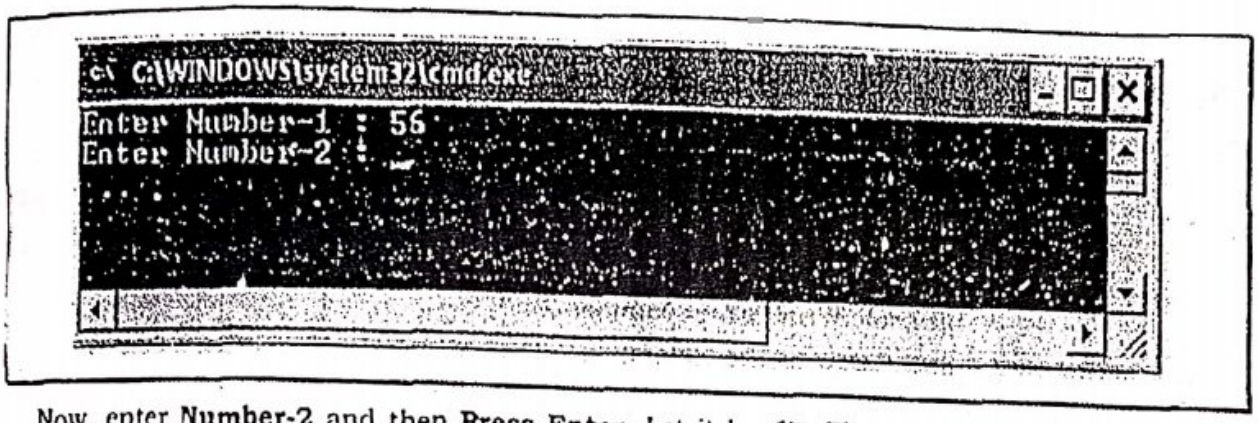
```
using System;
namespace RelationalOperators
{
    class Program
    {
        static void Main(string[] args)
        {
            // Declaration of required variables
```

```
        string str1, str2;
        int num1, num2;
        bool res;

        //Taking User Input as String
        Console.Write("Enter Number-1 : ");
        str1 = Console.ReadLine();
        Console.Write("Enter Number-2 : ");
        str2 = Console.ReadLine();

        //Converting Strings into integers
        num1 = Convert.ToInt32(str1);
        num2 = Convert.ToInt32(str2);

        //Working with various Relational operators.
        res = num1 < num2;
        Console.WriteLine("Result (num1<num2) is : {0}", res);
        res = num1 > num2;
        Console.WriteLine("Result (num1>num2) is : {0}", res);
        res = (num1 <= num2);
        Console.WriteLine("Result (num1<=num2) is : {0}", res);
        res = (num1 >= num2);
        Console.WriteLine("Result (num1>=num2) is : {0}", res);
        res = (num1 == num2);
        Console.WriteLine("Result (num1==num2) is : {0}", res);
        res = (num1 != num2);
        Console.WriteLine("Result (num1!=num2) is : {0}", res);
    }
}
```

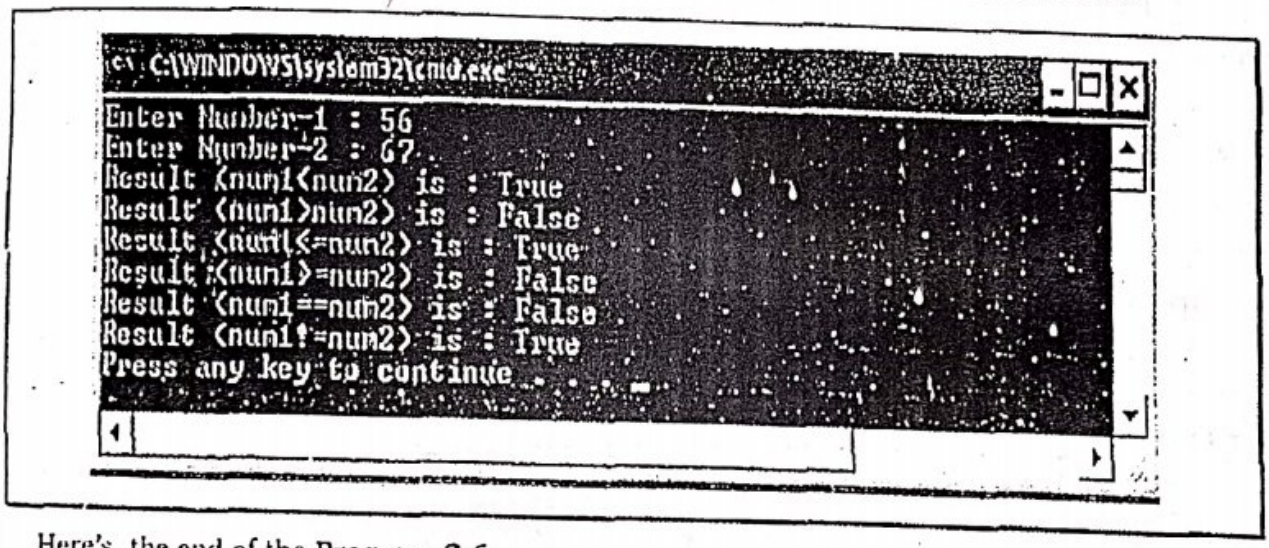
Step-5: Press Ctrl + F5. The output window will appear as follows:



Step-6: Now, enter **Number-1** and then **Press Enter**. Let it be **56.** Now, the output window will demand to enter **Number-2** and will be look like as follows:



Now, enter **Number-2** and then **Press Enter**. Let it be **67**. Then, the final result of various relational operations will be displayed on output window and will be look like as follows.



Here's, the end of the **Program 2-6**.

2.3.5.4 Logical Operators

Logical operators are used to perform test on two **Relational expressions**.

Syntax: <RelationalExpression1> **LogicalOperator** <RelationalExpression1>

It includes:

Operator	Description	Example

&&	Logical AND	int a = 30, b = 20, c = 40;
		bool res = (a > b) && (a > c);
		Value of **res** will be : **False**

||	Logical OR	int a 30, b = 20, c = 40;
		bool res = (a > b) || (a > c);
		Value of **res** will be : **True**

!	Logical NOT	bool res =! (true); Value of **res** will be : **False**

&&: Logical AND

Rules: Result will be true only if both the Boolean expressions are true, otherwise false.

Example:
```
int a=12;
bool res= a>10 && a<15;
```

Since both **a>10** and **a<15** are **True** expressions so the result of whole expression will be true i.e. value of **res** will be **True**.

||: Logical OR

Rules: Result will be true if any one of the Boolean expressions is true, otherwise false (if both the expressions are false).

Example:
```
int a=16;
bool res=a>10 || a<15;
```

Since **a>10** is **True** but **a<15** is **False** but the result of whole expression will be **True** because at least one expression is **True** i.e. value of **res** will be **True**.

! : Logical NOT

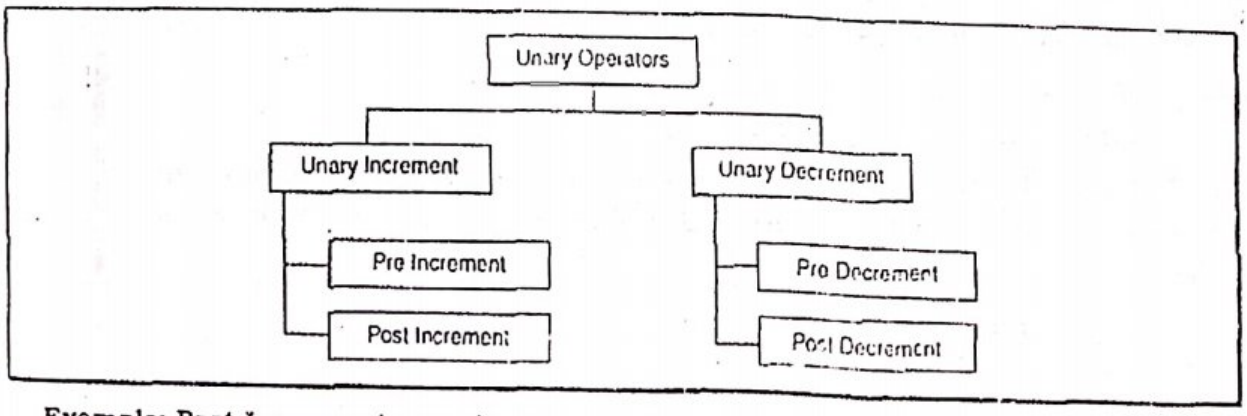
It just compliments the Boolean result of an expression.

Example:
```
int a=5;
bool x= !(a==5);
```

Here, **a==5** is a **True** expression but the '!' Operator will compliment this result and the result will be given as **False** i.e. value of **x** will be **False**.

2.3.5.5 Unary Operators

Operators that act upon a single operand are known as unary operators. There are two types of unary operators i.e. **Unary Increment** (++) and **Unary Decrement** (--). The unary increment operator causes its operand to be increased by 1, whereas the unary decrement operator causes tits operand to be decrease by 1. The operand used with each of these operators must be a single variable. These operators can be further subdivided as follows:



Example: Post Increment operators

```
int i=5;
i++;   //Post increment
```

Statement i++ is a post increment statement. The value of i will be incremented by 1 as a result of this statement. i.e. value of i will be 6 after the execution of the statement.

Example: Pre Increment operators

```
int i=5;
++i;    //Pre increment
```

Statement ++i is a pre increment statement. The value of i will be incremented by 1 as a result of this statement. i.e. value of i will be 6 after the execution of the statement.

Example: Difference between Post increment and Pre increment

Case-1:

```
int i=5;
int sum=++i;        //Pre increment
```

In this case, first, the value of 'i' will be incremented by 1 because of pre increment i.e. **'i'** will become 6. Then the value of 'i' will be assigned to variable **'sum'**. So, the result of the statement will be

i=6 and sum=6

Case-2:

```
int i=5;
int sum=i++;        //Post increment
```

In this case, first, the value of 'i' will be assigned to variable **'sum'**. i.e. the value of **'sum'** becomes 5. Then the value of 'i' will be incremented by 1. So, the result of the statement will be

sum=5 and i=6

Example: Post Decrement operators

```
int i=5;
i--;    //Post decrement
```

Statement i-- is a post decrement statement. The value of i will be decremented by 1 as a result of this statement. i.e. value of i will be 4 after the execution of the statement.

Example: Pre decrement operators

```
int i=5;
--i;    //Pre decrement
```

Statement --i is a pre decrement statement. The value of i will be decremented by 1 as a result of this statement. i.e. value of i will be 4 after the execution of the statement.

Example: Difference between Post decrement and Pre decrement

Case-1:

```
int i=5;
int res=--i;        //Pre decrement
```

In this case, first, the value of 'i' will be decremented by 1 because of pre decrement i.e. 'i' will become 4. Then the value of 'i' will be assigned to variable **'res'**. So, the result of the statement will be

i=4 and res=4

Scanned with OKEN Scanner

Case-2: int i=5;

int res=i++; **//Post decrement**

In this case, first, the value of 'i' will be assigned to variable **'res'**. i.e. the value of **'res'** becomes 5. Then the value of 'i' will be decremented by 1. So, the result of the statement will be

res=5 and **i=4**

2.3.5.6 Conditional Operator

Conditional operator is also known as **Ternary operator**. It is used to construct conditional expressions of the form

Expression1 ? Expression2 : Expression3 ;

Expression1 is evaluated first. If it is **true**, then the Expression2 will be evaluated and becomes the value of the expression. If Expression1 is **false**, Expression3 is evaluated and its value becomes the value of the expression.

Note: only one of the expressions (aeither Expression2 or Expression3) is evaluated.

For example, consider the following statements:

```
int num1=15;
int num2=25;
int greater=(num1 > num2) ? num1 : num2;
```

In this example, **greater** will be assigned the value of **num2**. Because Expression1 i.e. **num1>num2** is **false**.

2.3.5.7 Bitwise Operators

The bitwise operators are the operators which manipulates data at bit level. Bitwise operators may not be applied to **float** or **double** types. These operators work only on **char** and **int** types. There are following bitwise operators available in C#.

Operator	Description	Example
&	Bitwise AND	int res = 7&4; Value of **res** will be : **4**
|	Bitwise OR	int res = 7|4; Value of **res** will be : **7**
~	Bitwise Compliment	int res = ~ (5) ; Value of **res** will be : **–6**
^	Bitwise Exclusive OR	int res = 7^4; Value of **res** will be : **3**
<<	Bitwise Left Shift	int res = 7<<2; Value of **res** will be : **28**
>>	Bitwise Right Shift	int res = 7>>2; Value of **res** will be : **1**

2.3.6 Operators Precedence

Any time a program statement involves multiple operators, there has to be a set of rules that enables us to properly process the operators and operands in the expressions. Two operator characteristics determine how operands can be grouped with the operators. These characteristics are:

- Operator Precedence
- Operators Associativity

Precedence is the priority for grouping different types of operators with their operands. **Associativity** is the **left-to-right** or **right-to-left order** for grouping operands to operators that have the same precedence. An operator's precedence is meaningful only if other operators with higher or lower precedence are present. Expressions with higher-precedence operators are evaluated first. The grouping of operands can be forced by using parentheses. **Table 2-B** presents the precedence, or order, in which operators are processed.

Table 2-B

Operator Category	Operators	Associativity
Unary	+, –, !, ~, ++, —	right –to-left
Multiplicative	*, /, %	left–to-right
Additive	+, –	left–to-right
Shift	<<, >>	left–to-right
Relational	<, >, <=, >=	left–to-right
Equality	==, !=	left–to-right
Bitwise	&, ^, |	left–to-right
Logical	&&, ||	left–to-right
Conditional	?:	right-to-left
Assignment	+, +=, –=, *=, /=, %=, &=, |=, ^=, <<=, >>=	right-to-left

Example, in the following statements, the value **5** is assigned to both variable **a** and **b** because of the **right-to-left associativity** of the = operator. So, First, the value of variable **c** is assigned to variable **b** and then the value of variable **b** is assigned to variable **a**.

```
int b = 9;
int c = 5;
int a = b = c;
```

Here, the value of variable **a, b** and **c** will be **5**.

Lets take an another example to understand how operator precedence and associativity helpful to solve expressions. Let's take an expression to be evaluated, as follows:

```
x=a + b * c / d
```

In this statement, * and / operations are performed before + because of **precedence**. b is multiplied by c, before it is divided by d, this is because of **associativity**.

Scanned with OKEN Scanner

2.3.7 Flow Control Statements

C# executes statements in the order they are written. Control flow statements, break up the flow of execution by employing **Selection, Looping,** and **Branching.** These statements enable your program to conditionally execute particular block of code. These statements can be defined as follows:

2.3.7.1 Selection statements

- if statement
- if-else statement
- Ladder if else statement
- Nested if statement
- switch statement

a) if statement

The if statement evaluates an expression and if that evaluation is true then the specified action is taken. Its syntax is as follows:

Syntax:

```
if(Test_Condition)
{
    // Statements if Test Condition evaluates to True
}
```

Example: Check for equality.

```
if ( x ==y )
{
        Console.WriteLine(" x and y are equal");
}
```

Here, if the value of variable **'x'** is equal to the value of variable **'y'**, only then it will display **"x and y are equal"**

b) If else statement

This statement evaluates an expression and performs one action if that evaluation is true or a different action if it is false. Its syntax is as follows:

Syntax:

```
if(Test_Condition)
{
    // Statements if Test Condition evaluates to True
}
else
{
    // Statements if Test Condition evaluates to False
}
```

Example: Find greater among two numbers.

```
if ( num1>num2 )
{
Console.WriteLine("Number-1 is greater");
}
else
{
Console.WriteLine("Number-2 is greater");
}
```

Here, if the value of variable **'num1'** is greater than the value of variable **'num2'**, then the output will be displayed as **"Number-1 is greater"** and **else block** will be skipped, otherwise, the control will jump to the else block and the output will be displayed as **"Number-2 is greater".**

c) Ladder if else statement

It uses when there are multipath decisions are involved. Its syntax is as follows:

Syntax:

```
if(Test_Condition1)
        Statements Block - 1;
    else if(Test_Condition2)
            Statements Block - 2;
        else if(Test_Condition3)
                Statements Block - 3;
        .
        .
        .
        else
                Statement Block - n;
```

Example; Check whether the number is positive, negative or zero.

```
if ( num>0 )
{
    Console.WriteLine("Number is Positive");
}
 else if (num < 0)
{
    Console.WriteLine("Number is Negative");
}
else
{
    Console.WriteLine("Number is 0");
}
```

Scanned with OKEN Scanner

Here, if the value of variable **'num'** is greater than **0**, then the output will be displayed as **"Number is Positive"** and all other blocks will be skipped. Otherwise, the control will jump to the **else if** block, and if the value of variable **'num'** is less than **0**, then the output will be displayed as **"Number is Negative"** and all other blocks will be skipped. Otherwise, the control will jumps to the **else block**, and the output will be displayed as **"Number is 0"** and all other blocks will be skipped.

d) Nested if statement

It uses when there are series of decisions are involved. Its syntax is as follows:

Syntax:

```
if(Test_Condition1)
{
        //Statements if Test_Condition1 evaluates to True
        if (Test_Condition2)
        {
            //Statements if Test_Condition2 evaluates to True
        }
        else
        {
            //Statements if Test_Condition2 evaluates to False
        }
}
else
{
        //Statements if Test_Condition1 evaluates to False
        If (Test_Condition3)
        {
            //Statements if Test_Condition3 evaluates to True
        }
        else
        {
            //Statements if Test_Condition3 evaluates to False
        }
}
```

e) switch statement

It statement is a multi way decision statement. It is used for **Menu Driven** programming. Its syntax can be given as follows:

It tests the value of a given expression against a list of case values and when a match is found, a block of statements associated with that case is executed. There are some **rules** to be followed while using switch statement. Such are as follows:

Scanned with OKEN Scanner

- Expression must be of an integer type, character type, string type or constant expression.
- Case values must be constants or constant expressions, not variables.
- All the case values must be unique.
- Multiple cases are allowed with single block of code.
- It is mandatory to specify a break statement for each and every case block, without which the compiler will generate an error.

The use of switch statement can be demonstrated by the Program 2-7.

Program 2-7

Description of the Program 2-7: This is a Menu Driven program. Number of options will be given in menu. We will take input from the user to choose one of the option of the menu. Depending upon the user's choice further actions will be taken.

Step-1: Click on **File** menu and select **New >> Project.**

Step-2: In **New Project Dialog**, select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application**.

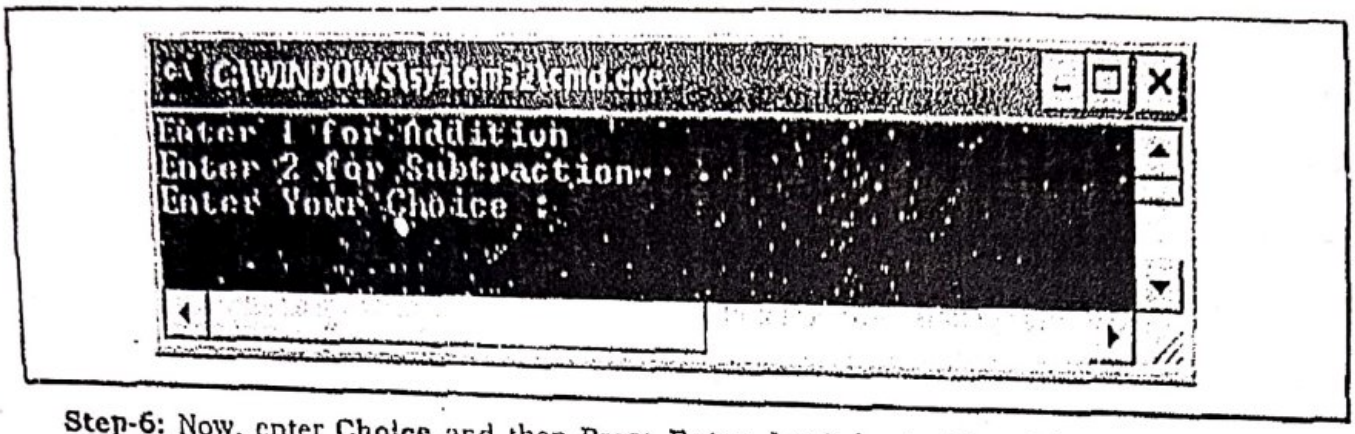
Step-3: Name the Application **"SwitchStatement"** and set location.

Step-4: Add the following code to the C# Console Application.

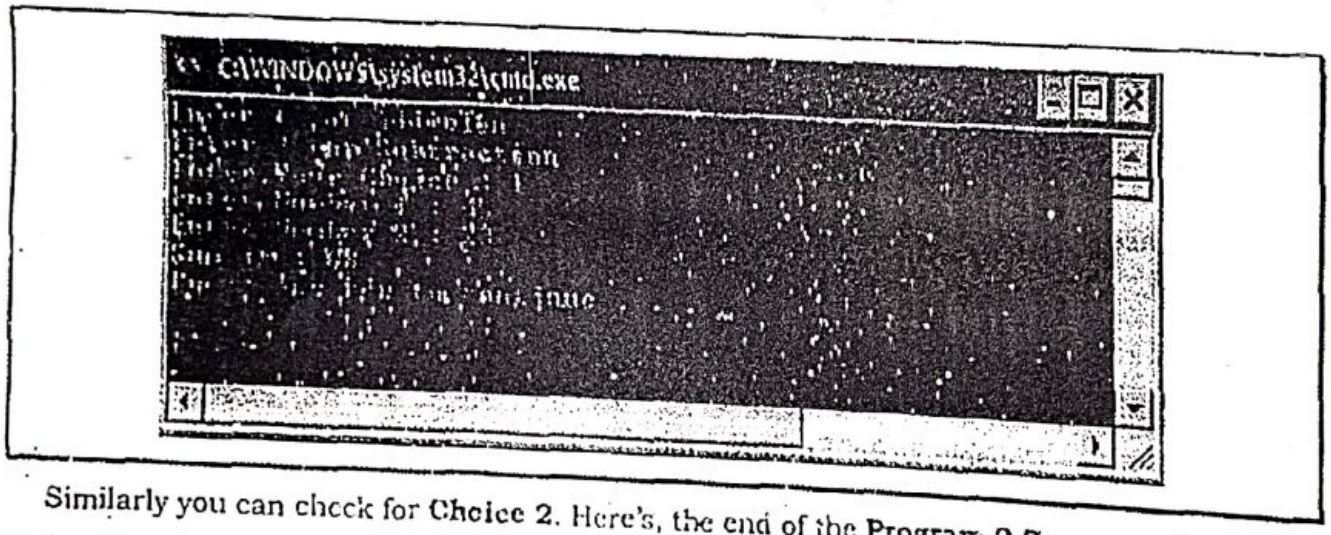
```
using System;
namespace SwitchStatement
{
  class Program
  {
    static void Main(string[] args)
    {
      //Declaration of required variables
      string str1, n1, n2;
      int choice, num1, num2, add,subtract;
      //Menu of various options
      Console.WriteLine("Enter 1 for Addition");
      Console.WriteLine("Enter 2 for Subtraction");
      //Taking User's choice as String
      Console.Write("Enter Your Choice : ");
      str1 = Console.ReadLine();
      //Converting String into integers
      choice = Convert.ToInt32(str1);
      //Working with switch statement
      switch (choice)
```

```
        {
            case 1:
                    //Taking User Input as String
                    Console.Write("Enter Number-1 : ");
                    n1 = Console.ReadLine();
                    Console.Write("Enter Number-2 : ");
                    n2 = Console.ReadLine();
                    //Converting Strings into integers
                    num1 = Convert.ToInt32(n1);
                    num2 = Convert.ToInt32(n2);
                    //Add num1 and num2, display result.
                    add = num1 + num2;
                    Console.WriteLine("Sum is : {0}", add);
                    break;
            case 2:
                    //Taking User Input as String
                    Console.Write("Enter Number-1 : ");
                    n1 = Console.ReadLine();
                    Console.Write("Enter Number-2 : ");
                    n2 = Console.ReadLine();
                    //Converting Strings into integers
                    num1 = Convert.ToInt32(n1);
                    num2 = Convert.ToInt32(n2);
                    //subtract num1 and num2, display result
                    subtract = num1 - num2;
                    Console.WriteLine("Result of subtraction is : {0}", subtract);
                    break;
            default:
                    Console.WriteLine("Invalid Choice!");
                    break;
        }
    }
}
```

Step-5: Press Ctrl + F5. The output window will appear as follows:



Step-6: Now, enter **Choice** and then **Press Enter.** Let it be **1.** After this, user has to enter **Number-1** and **Number-2** (45 and 23 respectively), and the sum of these numbers will be displayed on the screen as follows:



Similarly you can check for **Choice 2.** Here's, the end of the **Program 2-7.**

2.3.7.2 Looping Constructs

The process of repeatedly execution of a block of statements is called as looping. Loops are used to develop concise programs. it includes four steps:

- Setting and initialization of a counter.
- Execution of the statements in the loop.
- Test for a specified condition for execution of the loop.
- Incrementing or Decrementing the counter.

C# provides following types of loops:

a) while loop
b) do while loop
c) for loop

a) while loop

The **while** loop is an **entry control** loop. The while statement continually executes a block of statements while a particular condition is **true**. Its syntax is as follows:

Syntax:

```
initialization;
while (Test_Condition)
{
        statement(s) ;
        increment / decrement;
}
Statement-X;
```

The whilestatement evaluates Test Condition, which must return a boolean value. If the expression evaluates to **true**, the while statement executes the statement(s) in the while block. Otherwise, the control will jump to the **Statement-X**.

The while statement continues testing the expression and executing its block until the expression evaluates to **false**.

Example: To calculate Factorial of 5.

```
int num=5, factorial=1;
while (num>=1)
  {
        factorial=factorial * num;
        num--;
  }
Console.WriteLine("Factorial is : {0}",factorial);
```

b) do while loop

The **do while** loop is an **exit control** loop. The do while evaluates its expression at the bottom of the loop instead of the top. In do while loop, whether the test condition is true or false, block of statements within do, are always executed **at least once**. So, it is also used for menu driven programming. Its syntax is as follows:

Syntax:

```
initialization;
do
{
        statement(s) ;
        increment / decrement;
} while (Test_Condition);
Statement-X;
```

Example: To calculate Factorial of 5.

```
int num=5, factorial=1;
do
  {
        factorial=factorial * num;
```

```
            num--;
        } while (num>=1);
    Console.WriteLine("Factorial is : {0}",factorial);
```

c) for loop

It provides a compact way to iterate over a range of values.

Syntax:

```
for (initialization; termination; increment)
{
    statement(s);
}
```

Example: To calculate Factorial of 5.

```
int num, factorial=1;
for(num=5;num>=1;num--)
  {
    factorial=factorial * num;
  }
Console.WriteLine("Factorial is : {0}",factorial);
```

When using this version of the for statement, keep in mind that:

- The initialization expression initializes the loop. It executes only once, as the loop begins.
- When the termination expression evaluates to **false**, the loop terminates.
- The increment expression is invoked after each iteration through the loop.

2.3.7.3 Branching statements

It consists of **break, continue** and **return**.

a) break statement

Some times we need to exit from a loop before the completion of the loop then we use **break** statement and exit from the loop and loop is terminated. The break statement is used in **while** loop, **do while** loop, **for** loop and also used in the **switch** statement. A break statement causes an exit from the innermost containing while, do, for or switch statement.

Syntax:

```
break;
```

b) continue statement

The continue statement skips the current iteration of an innermost **for, while,** or **do while** loop and start the next iteration immediately. It can only be used with while, do while or for loop.

Syntax:

```
continue;
```

The use of break and continue statements can be given by the **Program 2-8.**

Scanned with OKEN Scanner

Program 2-8

Description of the Program 2-8: In this program we are going to display 1 to 10, but we will skip the iteration at value 5 and will break the iteration at value 9.

Step-1: Click on **File** menu and select **New >> Project.**

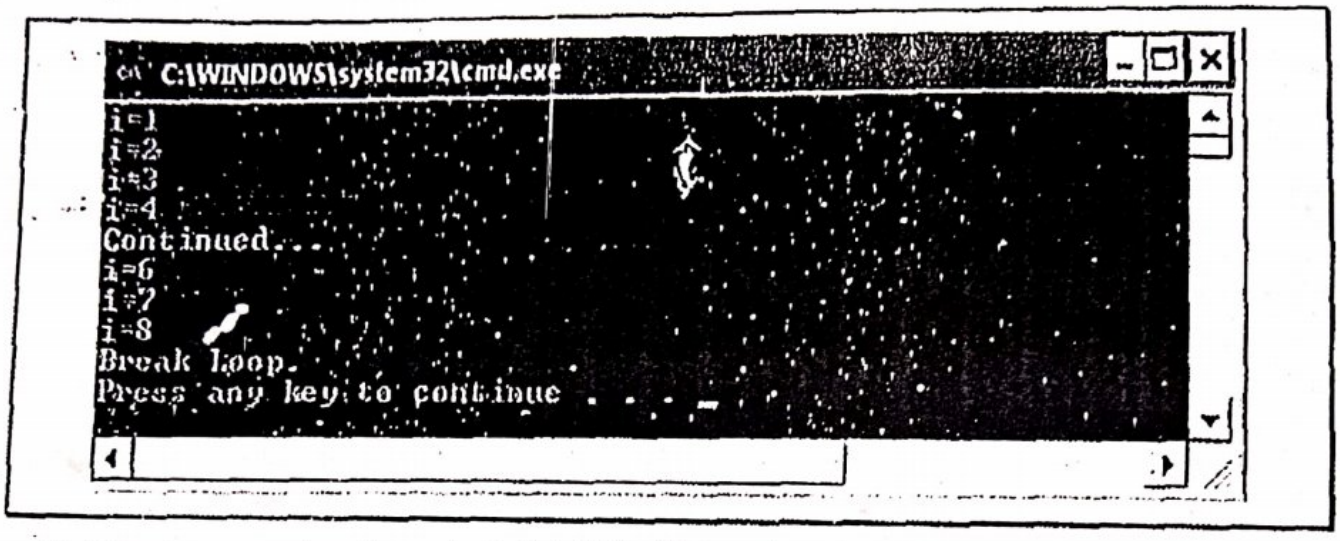
Step-2: In **New Project Dialog**, select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application.**

Step-3: Name the Application **"Branching"** and set location.

Step-4: Add the following code to the C# Console Application.

```
using System;
namespace Branching
{
   class Program
   {
      static void Main(string[] args)
      {
            int i;
            for (i=1;i<=10;i++)
            {
                  if (i == 5)
                  {
                        Console.WriteLine("Continued...");
                        continue;
                  }
                  else if (i == 9)
                  {
                        Console.WriteLine("Break Loop.");
                          break;
                  }
                  Console.WriteLine("i={0}", i);
            }
      }
   }
}
```

Step-5: Press Ctrl + F5. The output window will appear as follows:



In this program, when the value of variable **'i'** becomes 5 the loop continues with the next iteration which means that at i=5 the **continue** statement executed and the control will jumps to the increment statement (i++) of the loop. Similarly, when value of variable **'i'** becomes 9 the **break** statement will be executed and the control jumps out from the loop which means that loop will be terminated. Here's the end of **Program 2-8.**

c) return statement

The return statement used within a method. When this statement executed all the statements after return statement will be skipped because the return statement exits from the current method, and control flow returns to where the method was invoked (called). It has two forms, one that returns a value, and another that doesn't return any value. **For example:** To return a value, simply put the value (or an expression) after the return keyword as follows:

```
return Value;
```

The **data type** of the returned **Value** must match the **return type** of the method. When a method is declared void, use the form of **return** that doesn't return any value as follows:

```
return;
```

2.3.8 Arrays

An array is a collection of homogeneous elements. All the elements of an array share same name and same type. Elements of an array can be differentiated by their index. An array is used to make program concise and easy to read. C# arrays are a reference type. Each array in C# is an object and is inherited from System.Array class. An array can be declared as follows:

Syntax: DataType[] arrayName = new DataType[Size of Array];

Example: int[] marks = new int[5];

In this example name of the array is **marks** and the size of array is 5 which means array **marks** contains 5 elements indexed from 0 to 4 i.e. marks[0], marks[1], marks[2], marks[3] and marks[4]. All these elements are of type **int**.

Scanned with OKEN Scanner

The size of an array is fixed and must be defined before using it. However, you can use variables to define the size of the array as follows:

```
Example: int size=5;
         int[] marks = new int[size];
```

We can assign value to an array element as follows:

```
Example: marks[0]=10;
```

Initialization of an array can be done as follows:

```
Example:        int[] marks={10,20,30,40,50};
```

In this example name of the array is **marks** and the size of array will set to 5 because in the curly braces {} five elements are given. Values will be initialized as:

```
marks[0]=10
marks[1]=20
marks[2]=30
marks[3]=40
marks[4]=50
```

We can use loops to work with arrays. It make accessing of elements of an array easier. **For example**, suppose an array has **100** elements and we have to display all these elements. If we try to do this without any loop then either we have to use **WriteLine()** method **100 times** (one for each element) or we can do it in single **WriteLine()** method but with a complex statement. Now, if we use any loop to do so then this can be done in a very easy way as follows:

```
for(int index=0; index<100; index++)
{
    Console.WriteLine(ArrayName[index]);
}
```

As we mentioned earlier, C# arrays are a reference type and each array is an object and is inherited from **System.Array** class. This class has lot of useful **properties** and **methods** that can be applied to any instance of an array that we define. System.Array class has a very useful read-only property named **Length** that can be used to find the **length** or **size** of an array. We can use this property while accessing the elements of an array using any loop as follows:

```
for(int index=0; index<ArrayName.Length; index++)
{
    Console.WriteLine(ArrayName[index]);
}
```

The use of Array can be demonstrated by the Program 2-9.

Program 2-9

Description of the Program 2-9: In this program an integral array of 10 elements is initialized and then we will calculate the sum of these elements.

Step-1: Click on **File** menu and select **New >> Project.**

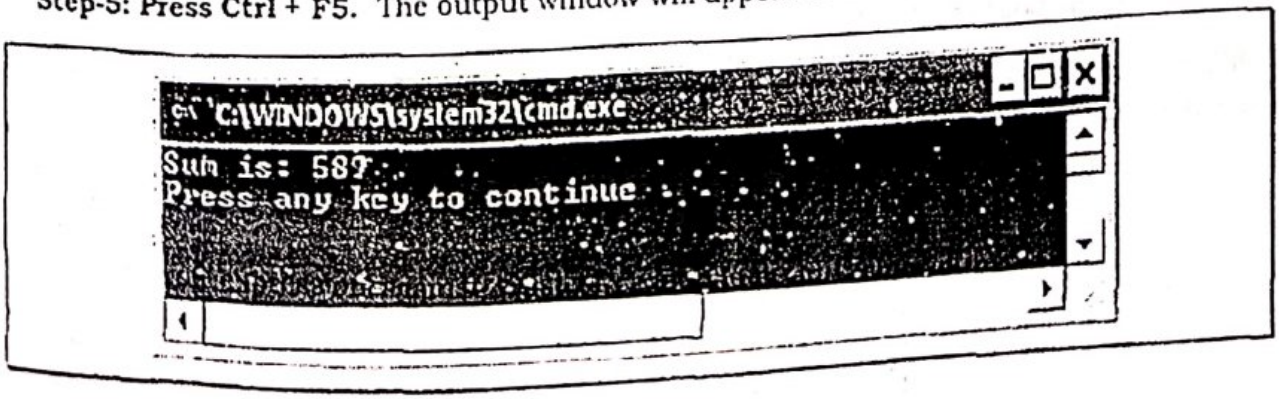
Step-2: In **New Project Dialog**, select **Project types** as **Visual C#** and from **Visual Studio Installed Templates** Select **Console Application.**

Step-3: Name the Application **"ArrayList"** and set location.

Step-4: Add the following code to the C# Console Application.

```
using System;
namespace ArrayList
{
    class Program
    {
        static void Main(string[] args)
        {
            //Initializing an array named marks with 10 elements
             int[] marks = { 50, 60, 45, 39, 74, 35, 43, 67, 77, 99 };

            //Initializing sum as 0
            int sum = 0;

            //calculating sum of all the elements of array marks
             for (int index = 0; index < marks.Length; index++)
             {
                    //Accessing an element of marks and add with sum
                      sum = sum + marks[index];
             }
            //Display sum of all array elements
             Console.WriteLine("Sum is: {0}", sum);
        }
    }
}
```

Step-5: Press Ctrl + F5. The output window will appear as follows:



REVIEW QUESTIONS

1. Describe the structure of typical C# program.
2. Why do we use the using directive in a C# program?
3. What is the importance of the Main method in a C# program?
4. Why do we use comments in a program?
5. Describe the differences between the two styles of comments used in C#.
6. What are component libraries? How are they different from executable application programs?
7. What is an alias? Where and how it is used?
8. How does the Write() method differ from the WriteLine() method?
9. Describe in detail the steps involved in implementing an application program in C#.
10. What are command line arguments? How are they useful?
11. How literals and variables are are important in developing a program?
12. Why do we need to use constants in a program?
13. List the two predefined reference types.
14. What do you mean by scope of a variable?
15. State the scope of the following variables.

    (a) Local variables (b) Field variables (c) Method parameters
16. What is initialization? Why is it necessary?
17. When dealing with a very small or very large numbers, what steps would you take to improve the accuracy of computations?
18. How do the value types differ from reference types in terms of their storage?
19. What does a declaration of a variable accomplish?
20. Define following

    (i) Literal (ii) Constant (iii) Variable
21. What is a mixed-mode arithmetic expression? How is it evaluated?
22. What is a shorthand assignment operator? What are the advantages of using a shorthand operator?
23. Why do we require type conversion operations?
24. What do you mean by implicit conversion? Give an example of application.
25. What do you mean by explicit conversion? Give an example of application.
26. Give an example of integer overflow. Will the same example work correctly, if we use the floating point values?

Scanned with OKEN Scanner

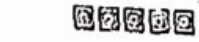
27. Give an example of floating point round-off error. How would you overcome this problem?
28. Given the value of a variable, write a C# statement (without using if construct) that will produce the absolute value of the variable.
29. What will be the output of the following code?

```
string s;
System.Console.WriteLine ("s =" + s);
```

30. What will be output of the following code snippet?

```
int x =10;
int y =20:
if ((x<y) || (x=5)>10)
Console.WriteLine(x); else
  Console.WriteLine(y);
```

31. What is the sequence of execution of Switch statement?
32. What are the various forms of if statement and their specific uses?
33. In what ways does a Switch statement differ from an if statement?



Scanned with OKEN Scanner